

Deep Learning Optimization methods

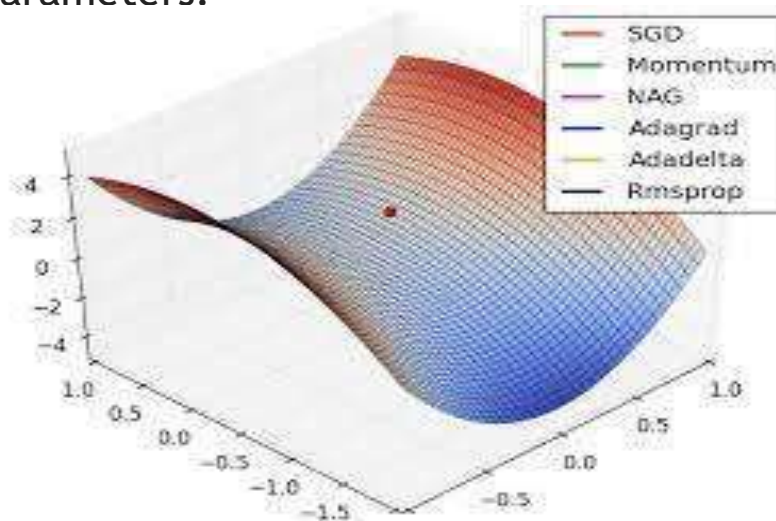
Omkar N

Contents

- ▶ What Are Optimizers in Deep Learning?
- ▶ Important Deep Learning Terms
- ▶ Gradient Descent Deep Learning Optimizer
- ▶ Stochastic Gradient Descent Deep Learning Optimizer
- ▶ Stochastic Gradient Descent With Momentum Deep Learning Optimizer
- ▶ Mini Batch Gradient Descent Deep Learning Optimizer
- ▶ Adagrad (Adaptive Gradient Descent) Deep Learning Optimizer
- ▶ RMS Prop (Root Mean Square) Deep Learning Optimizer
- ▶ Adam Optimizer in Deep Learning
 - Adam Optimizer Formula

What Are Optimizers in Deep Learning?

- ▶ Optimizer algorithms are optimization methods that help improve a deep learning model's performance.
- ▶ While training the deep learning optimizer model, modify each epoch's weights and minimize the loss function.
- ▶ An optimizer is a function or an algorithm that adjusts the attributes of the neural network, such as weights and learning rates.
- ▶ Thus, it helps in reducing the overall loss and improving accuracy.
- ▶ The problem of choosing the right weights for the model is a daunting task, as a deep learning model generally consists of millions of parameters.
- ▶ It raises the need to choose a suitable optimization.



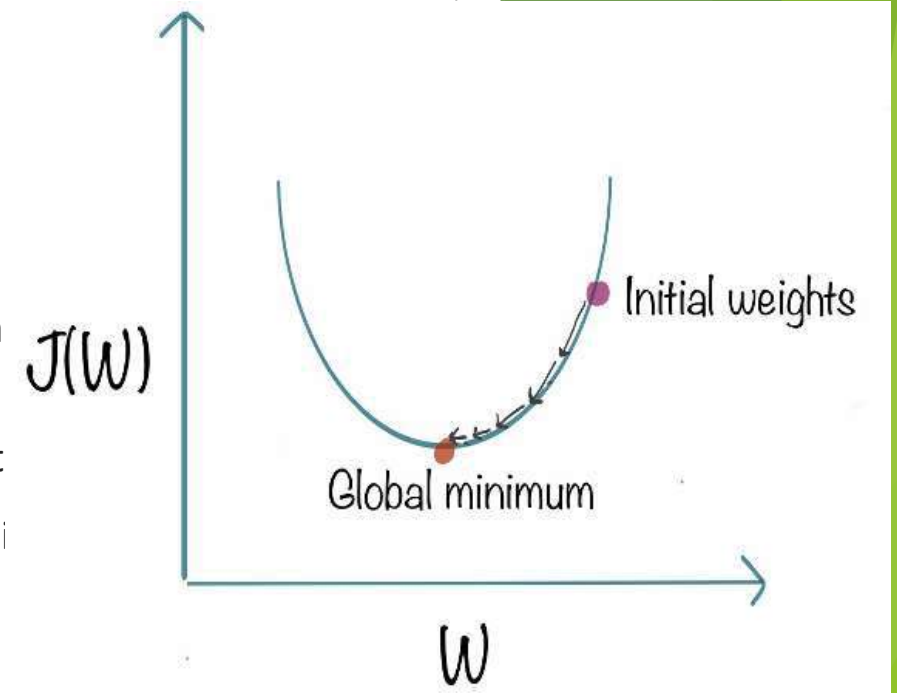
Gradient Descent Deep Learning Optimizer

Gradient Descent can be considered the popular kid among the class of optimizers. This optimization algorithm uses calculus to modify the values consistently and to achieve the local minimum

Gradient descent works as follows:

1. It starts with some coefficients, sees their cost, and searches for cost value lesser than what it is now.
 2. It moves towards the lower weight and updates the value of the coefficient
 3. The process repeats until the local minimum is reached. A local minimum is a point beyond which it can not proceed.
- Gradient descent works best for most purposes. However, it has some downsides too. It is expensive to calculate the gradients if the size of the data is huge.

$$x_{\text{new}} = x - \text{alpha} * f'(x)$$



Types of Gradient Descent:

Typically, there are three types of Gradient Descent:

- ▶ Batch Gradient Descent
- ▶ Stochastic Gradient Descent
- ▶ Mini-batch Gradient Descent

Batch Gradient Descent: Batch Gradient Descent involves calculations over the full training set at each step as a result of which it is very slow on very large training data. Thus, it becomes very computationally expensive to do Batch GD.

In SGD, instead of using the entire dataset for each iteration, only a single random training example (or a small batch) is selected to calculate the gradient and update the model parameters. This random selection introduces randomness into the optimization process, hence the term “stochastic” in stochastic Gradient Descent

Mini-batch gradient descent is a gradient descent modification that divides the training dataset into small batches that are used to compute model error and update model coefficients.

Batch Gradient Descent	Stochastic Gradient Descent	Mini-Batch Gradient Descent
Since the entire training data is considered before taking a step in the direction of gradient, therefore it takes a lot of time for making a single update.	Since only a single training example is considered before taking a step in the direction of gradient, we are forced to loop over the training set and thus cannot exploit the speed associated with vectorizing the code.	Since a subset of training examples is considered, it can make quick updates in the model parameters and can also exploit the speed associated with vectorizing the code.
It makes smooth updates in the model parameters	It makes very noisy updates in the parameters	Depending upon the batch size, the updates can be made less noisy – greater the batch size less noisy is the update

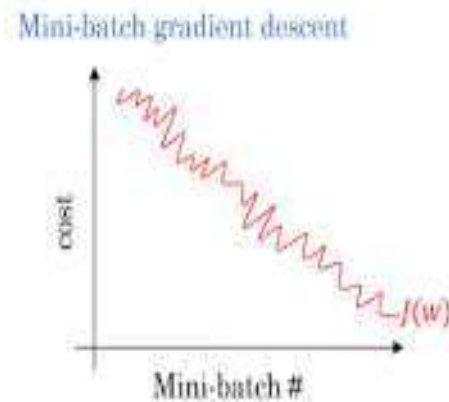
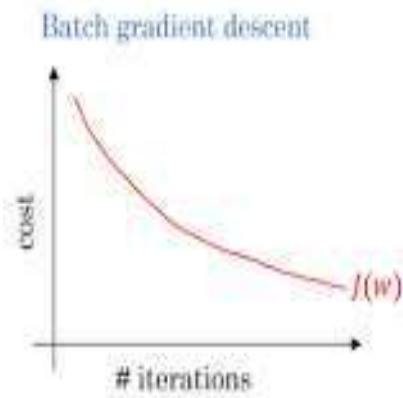
Gradient Descent With Momentum Deep Learning Optimizer

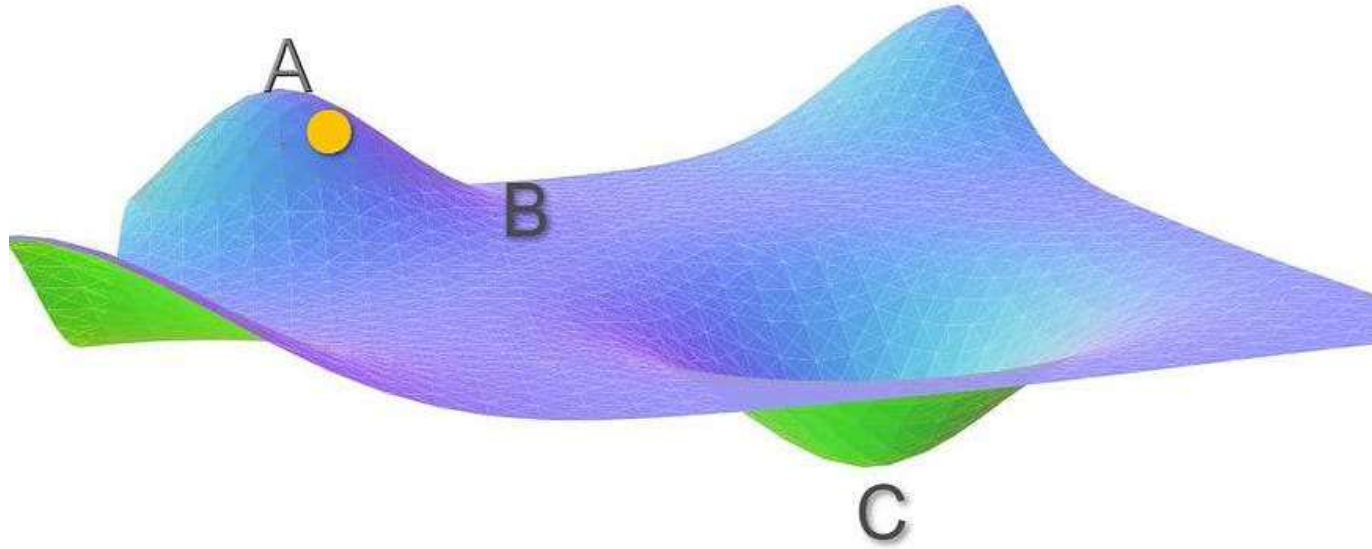
The problem with gradient descent is that the weight update at a moment (t) is governed by the learning rate and gradient at that moment only. It doesn't take into account the past steps taken while traversing the cost space.

$$\Delta w(t) = -\eta \delta(t)$$

It leads to the following problems:

1. The gradient of the cost function at saddle points(plateau) is negligible or zero, which in turn leads to small or no weight updates. Hence, the network becomes stagnant, and learning stops
2. The path followed by Gradient Descent is very jittery even when operating with mini-batch mode





- Let's assume the initial weights of the network under consideration correspond to point A.
- With gradient descent, the Loss function decreases rapidly along the slope AB as the gradient along this slope is high.
- But as soon as it reaches point B the gradient becomes very low.
- The weight updates around B is very small. Even after many iterations, the cost moves very slowly before getting stuck at a point where the gradient eventually becomes zero.
- In this case, ideally, cost should have moved to the global minima point C, but because the gradient disappears at point B, we are stuck with a sub-optimal solution.

Contd..

► **How can momentum fix this?**

Now, Imagine you have a ball rolling from point A. The ball starts rolling down slowly and gathers some **momentum** across the slope AB.

When the ball reaches point B, it has accumulated enough momentum to push itself across the plateau region B and finally following slope BC to land at the global minima C.

This can be done by using an Exponential Moving Average(EMA). An exponential moving average is a moving average that assigns a greater weight on the most recent values.

$$v(t) = \beta * v(t - 1) + (1 - \beta) * \delta(t)$$

$v(t)$: is the new weight update done at iteration t

β : Momentum constant

$\delta(t)$: is the gradient at iteration t

Adagrad (Adaptive Gradient Descent) Deep Learning Optimizer

- The idea behind Adagrad is to use different learning rates for each parameter base on iteration. The reason behind the need for different learning rates is that the learning rate for sparse features parameters needs to be higher compare to the dense features parameter because the frequency of occurrence of sparse features is lower.

$$\text{SGD} \Rightarrow w_t = w_{t-1} - \eta \frac{\partial L}{\partial w_{t-1}}$$

$$\text{Adagrad} \Rightarrow w_t = w_{t-1} - \eta'_t \frac{\partial L}{\partial w_{t-1}}$$

$$\text{where } \eta'_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

ε is a small +ve number to avoid divisibility by 0

$$\alpha_t = \sum_{i=1}^t \left(\frac{\partial L}{\partial w_{t-1}} \right)^2 \text{ summation of gradient square}$$

Contd..

- ▶ The benefit of using Adagrad is that it abolishes the need to modify the learning rate manually. It is more reliable than gradient descent algorithms and their variants, and it reaches convergence at a higher speed.
- ▶ One downside of the AdaGrad optimizer is that it decreases the learning rate aggressively and monotonically.
- ▶ There might be a point when the learning rate becomes extremely small.
- ▶ This is because the squared gradients in the denominator keep accumulating, and thus the denominator part keeps on increasing.
- ▶ Due to small learning rates, the model eventually becomes unable to acquire more knowledge, and hence the accuracy of the model is compromised

RMS Prop (Root Mean Square) Deep Learning Optimizer

- ▶ Like other gradient descent algorithms, RMSprop works by calculating the gradient of the loss function with respect to the model's parameters and updating the parameters in the opposite direction of the gradient to minimize the loss. However, RMSProp introduces a few additional techniques to improve the performance of the optimization process.
- ▶ One key feature is its use of a moving average of the squared gradients to scale the learning rate for each parameter. This helps to stabilize the learning process and prevent oscillations in the optimization trajectory.
- ▶ The algorithm can be summarized by the following RMSProp formula:
- ▶ $v_t = \text{decay_rate} * v_{t-1} + (1 - \text{decay_rate}) * \text{gradient}^2$
parameter = parameter - learning_rate * gradient / ($\sqrt{v_t} + \text{epsilon}$)

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) \left(\frac{\delta C}{\delta w} \right)^2$$

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{E[g^2]_t}} \frac{\delta C}{\delta w}$$

Adam Optimizer in Deep Learning

- ▶ Adam optimizer, short for Adaptive Moment Estimation optimizer, is an optimization algorithm commonly used in deep learning. It is an extension of the stochastic gradient descent (SGD) algorithm and is designed to update the weights of a neural network during training.
- ▶ The name “Adam” is derived from “adaptive moment estimation,” highlighting its ability to adaptively adjust the learning rate for each network weight individually. Unlike SGD, which maintains a single learning rate throughout training, Adam optimizer dynamically computes individual learning rates based on the past gradients and their second moments.

$$\begin{aligned}m_t &= \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t \\v_t &= \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\w_{t+1} &= w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t\end{aligned}$$